Now we can clearly see how `sort()` is really functioning: by comparing byte values. `.-Orange` starts with the lowest value byte, *0x2e*, and is placed first in the array. Next comes `0-banana`, which begins with *0x30*, and so on.

So far, so good. Now let's see what happens when we throw some non-Latin, multi-byte strings into the mix…

```
$polishNames = [
    "Łukasz",
    "Marcin",
    "Tomasz",
];
sort($polishNames);
print_r($polishNames);
var_dump(bin2hex("Tomasz"));
var_dump(bin2hex("Łukasz"));
```

```
Output:
Array
(
    [0] => Marcin
    [1] => Tomasz
    [2] => Łukasz
)
string(12) "546f6d61737a"
string(14) "c581756b61737a"
```

For those of you not familiar with the Polish alphabet, the order of the sorted array is actually incorrect. The character Ł lies between `L` and `M` in the Polish alphabet but sits at Unicode codepoint U+0141 (Latin Capital Letter L with Stroke), which is quite a way beyond U+004C (`L`) and U+004D (`M`). Furthermore, the UTF-8 code unit for Ł is *0xC5 0x81*. PHP's `sort()` function knows nothing of Polish or Unicode and just blindly organizes the strings by byte value. *0x54* is a lower value than *0xC5*, so `Tomasz` is placed before `Łukasz` in the ordered array. There's no special treatment of strings here; in order for the byte values to have any meaning as a natural language, we would need to apply some regional collation rules. The good news is PHP has some useful features for doing exactly that (as you should expect from a language specializing in processing text for the world's websites), which we will examine in detail later on in the chapter.